< interview_prep.js />

Frontend Developer

60 Questions · 5 Coding Tasks · Revision Strategy

HTML5 CSS3 JavaScript DOM ES6+ Async/Await

= High Priority | All answers beginner-friendly | Revise in 1 day

■ Section 1 : HTML

5 Questions · Fundamentals

Q1 ■ **What is the difference between and ?**

→ is a block-level element (takes full width, starts new line). is an inline element (only as wide as content). Use for layout sections, for styling small text parts.

#html

Q2 **What is semantic HTML? Give 3 examples.**

→ Semantic tags clearly describe their meaning. Examples: <header> (page header), <article> (independent content), <nav> (navigation links). Helps SEO and accessibility.

#semantic

Q3 ■ **What is the difference between id and class attributes?**

→ id is unique – only one element per page can have a specific id. class can be reused on multiple elements. id has higher CSS specificity than class.

#html

Q4 **What are data-* attributes used for?**

→ Custom attributes to store extra data on HTML elements without using non-standard attributes. Example: <button data-id='5'>. Access via element.dataset.id in JS.

#html

Q5 ■ **What is the difference between <script>, <script async> and <script defer>?**

→ Normal <script> blocks HTML parsing. async downloads in parallel, executes immediately (order not guaranteed). defer downloads in parallel, executes after HTML is parsed (order maintained). Prefer defer for most scripts.

#performance

■ Section 2 : CSS

10 Questions · Styling & Layout

Q6 ■ What is the CSS Box Model?

→ Every element is a box: Content → Padding → Border → Margin (outside to inside). box-sizing: border-box includes padding+border in the width, making layouts easier.

#boxmodel

Q7 ■ Difference between display: flex and display: grid?

→ Flexbox is 1-dimensional (row OR column). Grid is 2-dimensional (rows AND columns simultaneously). Use Flex for navbars/cards in a row; Grid for full-page layouts.

#layout

Q8 ■ What is CSS specificity? How is it calculated?

→ Priority order: inline styles (1000) > #id (100) > .class / [attr] / :pseudo-class (10) > element (1). Higher specificity wins. !important overrides all.

#specificity

Q9 ■ What is the difference between position: relative, absolute, fixed, sticky?

→ relative: offset from normal position. absolute: offset from nearest positioned ancestor. fixed: relative to viewport (stays on scroll). sticky: relative until scroll threshold, then fixed.

#positioning

Q10 How does z-index work?

→ Controls stacking order of positioned elements (not static). Higher z-index = on top. Works only on elements with position set (relative/absolute/fixed/sticky).

#zindex

Q11 What are CSS pseudo-classes and pseudo-elements? Give examples.

→ Pseudo-classes select state: :hover, :focus, :nth-child(). Pseudo-elements style part of element: ::before, ::after, ::first-letter. Classes use single colon, elements use double colon (CSS3).

#pseudo

Q12 ■ What is the difference between em, rem, px, vw, vh?

→ px: fixed pixels. em: relative to parent font-size. rem: relative to root (html) font-size. vw/vh: % of viewport width/height. Prefer rem for scalable typography.

#units

Q13 How do CSS Flexbox align-items and justify-content differ?

→ justify-content controls main axis (row direction by default). align-items controls cross axis (column direction). Think: justify = horizontal, align = vertical (in default flex-direction: row).

#flexbox

Q14 ■ **What is a CSS media query? Write an example.**

→ Used for responsive design based on device conditions. Example: @media (max-width: 768px) { .container { flex-direction: column; } } — changes layout on mobile screens.

#responsive

Q15 **What is CSS specificity conflict resolution?**

→ When two rules have equal specificity, the one that appears LAST in the stylesheet wins (cascade). !important breaks this but should be avoided as it creates hard-to-debug issues.

#cascade

■ Section 3 : JavaScript

30 Questions · MAIN FOCUS — Closures, Hoisting, Event Loop, Promises, ES6+

Q16 ■ What is Hoisting in JavaScript? ■

→ JS moves var declarations and function declarations to the top of their scope before execution. var is hoisted & initialized as undefined. let/const are hoisted but NOT initialized (Temporal Dead Zone). Function declarations are fully hoisted.

#hoisting

Q17 What is the Temporal Dead Zone (TDZ)?

→ The period between entering a scope and the let/const declaration. Accessing the variable here throws ReferenceError. Example: console.log(x); let x = 5; — throws TDZ error.

#hoisting

Q18 ■ What is a Closure? ■

→ A closure is a function that remembers variables from its outer scope even after that scope has finished executing. Like a backpack the inner function carries. Used in data privacy, factory functions, event handlers.

#closure

Q19 ■ Give a real example where Closure is useful.

→ Counter example: function makeCounter(){ let count=0; return function(){ return ++count; }; } — each call to makeCounter() gives an independent counter. count stays private.

#closure

Q20 ■ What does 'this' refer to in JavaScript? ■

→ Depends on how function is called: global scope → window/undefined(strict). Object method → the object. Arrow function → outer/lexical this. Constructor → new object. Can be explicitly set with call(), apply(), bind().

#this

Q21 ■ What is the difference between call(), apply(), and bind()?

→ call(obj, arg1, arg2): invokes immediately with individual args. apply(obj, [arg1,arg2]): invokes immediately with array. bind(obj, arg1): returns NEW function with this bound. Remember: bind = later, call/apply = now.

#this

Q22 ■ Explain the JavaScript Event Loop. ■

→ JS is single-threaded. Has: Call Stack (executes sync code), Web APIs (handle async like setTimeout/fetch), Callback Queue (macrotasks), Microtask Queue (Promises). Event loop checks: if stack empty → run all microtasks → then one macrotask → repeat.

#eventloop

Q23 ■ **What is the difference between microtasks and macrotasks?**

→ Microtasks: Promise.then(), queueMicrotask(), MutationObserver — run BEFORE next macrotask. Macrotasks: setTimeout, setInterval, I/O callbacks — run after all microtasks are cleared. Microtasks always have priority.

#eventloop

Q24 ■ **What will this print: setTimeout(()=>console.log('A'),0); Promise.resolve().then(()=>console.log('B')); console.log('C');**

→ Output: C → B → A. C is sync (runs first). B is microtask (runs before macrotask). A is macrotask (setTimeout). Even with 0ms delay, macro runs after micro.

#eventloop

Q25 ■ **What is a Promise? What are its states? ■**

→ Object representing future value of async operation. States: Pending (initial), Fulfilled (success → .then()), Rejected (error → .catch()). Once settled, state doesn't change. Created with new Promise((resolve, reject) => {...}).

#promise

Q26 ■ **What is async/await and how does it relate to Promises? ■**

→ async/await is syntactic sugar over Promises. async function always returns a Promise. await pauses execution until Promise resolves. Makes async code look synchronous. Must use try/catch for error handling with await.

#async

Q27 ■ **What is Promise.all() vs Promise.allSettled() vs Promise.race()?**

→ Promise.all(): waits for all, fails fast if any rejects. Promise.allSettled(): waits for all regardless of rejection, returns all results. Promise.race(): resolves/rejects as soon as FIRST promise settles.

#promise

Q28 ■ **How to handle errors in async/await?**

→ Use try-catch-finally: try { const data = await fetchData(); } catch(err) { console.error(err); } finally { setLoading(false); } — finally always runs. Or .catch() on the returned promise.

#async

Q29 ■ **Explain map(), filter(), and reduce() with examples. ■**

→ map(): transforms each element, returns new array of same length. filter(): keeps elements matching condition, returns smaller array. reduce(): accumulates values into single result. All are non-mutating (don't change original).

#arrays

Q30 **What is the difference between map() and forEach()?**

→ map() returns a NEW array with transformed values. forEach() returns undefined — used only for side effects. Use map() when you need transformed data, forEach() for logging/DOM updates.

#arrays

Q31 ■ **How does Array.reduce() work? Write an example.**

→ reduce(callback, initialValue) where callback gets (accumulator, currentValue). Example: [1,2,3,4].reduce((sum, n) => sum + n, 0) returns 10. Can flatten arrays, group objects, count occurrences.

#arrays

Q32 **What is the difference between find() and filter()?**

→ filter() returns ALL matching elements as array. find() returns only the FIRST matching element (not array). If nothing found: filter returns [], find returns undefined.

#arrays

Q33 ■ **What are the main ES6 features? ■**

→ let/const, Arrow functions, Template literals `${var}`, Destructuring, Spread/Rest operators, Default parameters, Classes, Modules (import/export), Promises, Map/Set, Symbol, for...of loop.

#es6

Q34 ■ **What is Destructuring in JavaScript?**

→ Unpacking values from arrays/objects into variables. Array: const [a,b] = [1,2]. Object: const {name, age} = person. Can rename: const {name: myName} = person. Can set defaults: const {age=18} = person.

#es6

Q35 ■ **What is the difference between Spread (...) and Rest (...) operators?**

→ Same syntax, different use. Spread EXPANDS array/object: [...arr1, ...arr2]. Rest COLLECTS remaining elements: function sum(...nums). Spread appears in expressions, Rest appears in function parameters.

#es6

Q36 ■ **What are Arrow Functions and how do they differ from regular functions?**

→ Shorter syntax: const add = (a,b) => a+b. Key difference: arrow functions DON'T have their own 'this' — they inherit from lexical (outer) scope. No 'arguments' object. Can't be used as constructors.

#es6

Q37 ■ **What is optional chaining (?.) and nullish coalescing (??)?**

→ Optional chaining: user?.address?.city — returns undefined instead of throwing error if any part is null/undefined. Nullish coalescing: value ?? 'default' — uses default only if value is null/undefined (not 0 or '').

#es6

Q38 ■ **What is the difference between var, let, and const? ■**

→ var: function-scoped, hoisted, re-declarable. let: block-scoped, not re-declarable, has TDZ. const: block-scoped, must initialize, can't reassign (but object properties CAN change). Use const by default, let when reassignment needed.

#scope

Q39 What is a Prototype in JavaScript?

→ Every JS object has an internal [[Prototype]] link to another object. When property not found, JS looks up the prototype chain. Object.prototype is the top. This enables inheritance without classes. __proto__ or Object.getPrototypeOf() to access.

#prototype

Q40 ■ What is the difference between == and ===?

→ == (loose equality) does type coercion: '5' == 5 is true. === (strict equality) checks type AND value: '5' === 5 is false. Always use === to avoid bugs from implicit type conversion.

#types

Q41 What is a pure function?

→ A function that: 1) Given same input, always returns same output. 2) Has NO side effects (doesn't modify external state). Example: const add = (a,b) => a+b. Easier to test and reason about.

#functional

Q42 ■ What is debouncing vs throttling?

→ Debounce: delays execution until after user STOPS triggering (search input — wait till typing stops). Throttle: limits execution to once per time interval (scroll events — max once per 100ms). Both optimize performance.

#performance

Q43 What is the difference between null and undefined?

→ undefined: variable declared but not assigned. null: intentional absence of value (explicitly set). typeof undefined = 'undefined'. typeof null = 'object' (famous JS bug). Use null to intentionally empty a value.

#types

Q44 ■ What is event bubbling and event capturing?

→ Bubbling: event fires on target, then bubbles UP to ancestors (default). Capturing: event fires from root DOWN to target. stopPropagation() stops bubbling. addEventListener(event, fn, true) enables capturing. Most events bubble.

#events

Q45 ■ What is a higher-order function?

→ A function that takes another function as argument or returns a function. Examples: map(), filter(), reduce(), setTimeout(). Enables functional programming patterns. Closures + HOFs are core JavaScript patterns.

#functional

■ Section 4 : DOM & Browser

10 Questions · Web APIs & Browser Concepts

Q46 ■ What is the DOM? How is it different from HTML?

→ DOM (Document Object Model) is a live, tree-structured JS representation of the HTML. HTML is static markup text; DOM is the parsed, interactive object tree in memory. JS manipulates the DOM, not the raw HTML file.

#dom

Q47 ■ What is event delegation? Why is it useful?

→ Instead of adding listeners to each child, add ONE listener to parent. Use event.target to identify which child was clicked. More memory-efficient, works for dynamically added elements. Common pattern for lists/tables.

#events

Q48 ■ What is the difference between localStorage, sessionStorage, and cookies?

→ localStorage: persists until manually cleared, 5-10MB, same origin. sessionStorage: cleared when tab closes, 5MB, same tab only. Cookies: sent to server with every request, ~4KB, can have expiry date. Use localStorage for user preferences.

#storage

Q49 ■ What is the difference between innerHTML, textContent, and innerText?

→ innerHTML: gets/sets HTML including tags (XSS risk!). textContent: raw text including hidden elements, fastest. innerText: visible text only (respects CSS display:none). Use textContent for plain text to avoid XSS.

#dom

Q50 How does document.querySelector differ from getElementById?

→ querySelector uses CSS selectors (more flexible): document.querySelector('.class'), querySelector('#id'), querySelector('input[type=email]'). getElementById only gets by id but is slightly faster. querySelector returns first match, querySelectorAll returns NodeList.

#dom

Q51 What is the difference between window and document?

→ window is the global object — contains everything (document, location, history, alert, setTimeout). document is the DOM — represents the HTML content. window.document === document. window is the outermost container.

#browser

Q52 ■ What is CORS and why does it occur?

→ Cross-Origin Resource Sharing — browser security policy blocking requests from one origin (domain+port+protocol) to another. Server must send Access-Control-Allow-Origin header. Common error when frontend (localhost:3000) calls API on different port/domain.

#browser

Q53 ■ What is the difference between repaint and reflow?

→ Reflow (Layout): recalculates positions/sizes when geometry changes — expensive. Repaint: redraws pixels without geometry change (color, visibility) — cheaper. Minimize DOM manipulation, batch style changes, use transform/opacity for animations (no reflow).

#performance

Q54 ■ What does fetch() return? How do you handle errors with it?

→ fetch() returns a Promise that resolves to Response object even for 4xx/5xx errors (only rejects on network failure). Must check response.ok or response.status. Then call response.json() (also async). Use try/catch or .catch() for network errors.

#api

Q55 What is the difference between document.DOMContentLoaded and window.load?

→ DOMContentLoaded fires when HTML is parsed and DOM is ready (scripts deferred run first) — no waiting for images/CSS. window.load fires after everything (images, stylesheets, scripts) is fully loaded. Use DOMContentLoaded for most initialization.

#browser

■ Section 5 : Projects & Debugging

3 Scenario Questions

Q56 ■ Your fetch() call is failing silently. How do you debug it? ■

→ 1) Check Network tab in DevTools for request status & response. 2) Check Console for errors. 3) Verify URL is correct. 4) Check CORS headers in Response. 5) Confirm response.ok before .json(). 6) Add explicit .catch() to promise chain.

#debugging

Q57 ■ You added an event listener inside a loop and all items show the last value. Why?

→ Classic closure-in-loop bug with var. var is function-scoped, so all callbacks share the same i. Fix: use let (block-scoped), or IIFE, or .forEach(). let creates a new binding per iteration.

#closure

Q58 ■ Your page is slow. How do you diagnose and fix performance issues?

→ 1) Lighthouse audit in DevTools. 2) Check Network tab — large assets, waterfall. 3) Check Performance tab — long tasks, reflows. Fixes: lazy loading images, code splitting, debounce events, use CSS transforms not top/left, minimize DOM operations, cache API calls.

#performance

■ Section 6 : HR & Behavioral

2 Questions · Soft Skills

Q59 ■ Tell me about a project you built and a challenge you faced.

→ Structure: 1) Briefly describe the project and tech stack. 2) Mention ONE specific technical challenge (bug, API issue, design problem). 3) Explain how you solved it. 4) What you learned. Be specific — 'I debugged a CORS error by...' beats vague answers.

#hr

Q60 Where do you see yourself in 2 years as a frontend developer?

→ Good answer: 'I want to become proficient in React/framework, contribute to real product features, and grow into a mid-level role. I enjoy building user-facing features and want to deepen my JavaScript fundamentals while learning state management and testing.'

#hr

■ Real-World Coding Tasks

5 Practical Problems with Solutions

Task 1 ■ — Fetch API Data & Display

Problem: Fetch users from https://jsonplaceholder.typicode.com/users and display their names in a list.

```
async function loadUsers() {
  const list = document.getElementById('userList');
  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/users');
    if (!response.ok) throw new Error(`HTTP error: ${response.status}`);
    const users = await response.json();
    users.forEach(user => {
      const li = document.createElement('li');
      li.textContent = user.name;
      list.appendChild(li);
    });
  } catch (err) {
    list.innerHTML = `<li>Error: ${err.message}</li>`;
  }
}
loadUsers();
```

Key concepts: async/await, try/catch, response.ok check, DOM manipulation

Task 2 ■ — Form Validation

Problem: Validate an email input — show error if invalid, success if valid.

```
function validateEmail(email) {
  const regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return regex.test(email);
}

document.getElementById('emailInput').addEventListener('blur', function() {
  const msg = document.getElementById('errorMsg');
  if (!validateEmail(this.value)) {
    msg.textContent = '■ Invalid email address';
    msg.style.color = 'red';
  } else {
    msg.textContent = '■ Valid email!';
    msg.style.color = 'green';
  }
});
```

Key concepts: Regex, event listeners, DOM manipulation, validation pattern

Task 3 ■ — Event Delegation on Dynamic List

Problem: Add delete functionality to a dynamic list using event delegation.

```
// HTML: <ul id="todoList"></ul> <button id="addBtn">Add Item</button>

let count = 0;
document.getElementById('addBtn').addEventListener('click', () => {
```

```
  const li = document.createElement('li');
  li.innerHTML = `Item ${++count} <button class="del-btn" data-id="${count}">Delete</button>`;
  document.getElementById('todoList').appendChild(li);
});

// ONE listener for all delete buttons (event delegation)
document.getElementById('todoList').addEventListener('click', (e) => {
  if (e.target.classList.contains('del-btn')) {
    e.target.parentElement.remove();
  }
});
```

Key concepts: Event delegation, data attributes, dynamic DOM, e.target

Task 4 — Implement Debounce for Search Input

Problem: Only call search API after user stops typing for 500ms.

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}

const searchAPI = (query) => {
  console.log('Searching for:', query);
  // fetch('/api/search?q=' + query)
};

const debouncedSearch = debounce(searchAPI, 500);
document.getElementById('searchInput').addEventListener('input', (e) => {
  debouncedSearch(e.target.value);
});
```

Key concepts: Closure, debounce pattern, clearTimeout, ...args spread

Task 5 ■ — Promise Chain vs Async/Await

Problem: Show same logic in both Promise chain and async/await style.

```
// --- Promise Chain ---
fetch('/api/user')
  .then(res => res.json())
  .then(user => fetch(`/api/posts/${user.id}`))
  .then(res => res.json())
  .then(posts => console.log(posts))
  .catch(err => console.error(err));

// --- Async/Await (cleaner) ---
async function getUserPosts() {
  try {
    const userRes = await fetch('/api/user');
    const user = await userRes.json();
    const postsRes = await fetch(`/api/posts/${user.id}`);
    const posts = await postsRes.json();
```

```
    console.log(posts);
  } catch (err) {
    console.error(err);
  }
}
getUserPosts();
```

Key concepts: Promise chaining, async/await equivalence, error handling

■ Last Day Revision Strategy for Interview

How to revise everything in 1 day

■ Time	■ What to Do	■ Focus
Morning 6–8 AM	Skim HTML (5 Qs) + CSS (10 Qs) Revise Box Model, Flexbox, Grid, Specificity	Read answers, don't memorize — understand
Morning 8–11 AM	JavaScript: Closures, Hoisting, Scope Event Loop, Promises, async/await (Q16–Q28)	■ starred questions FIRST
Lunch 11–12 PM	Light break — watch 1 YouTube short on Event Loop (Visualize, don't just read)	Philip Roberts: What the heck is event loop?
Afternoon 12–2 PM	JS continued: Array methods, this, ES6 Dom & Browser (Q46–Q55) (Q29–Q55)	Write map/filter/reduce from memory once
Afternoon 2–4 PM	Coding Tasks 1–5 Type them out yourself (don't just read!) Focus on fetch + event delegation	Practice > passive reading
Evening 4–5 PM	Projects & Debugging scenarios HR answers — practice saying out loud	Mock interview with yourself
Evening 5–6 PM	Revisit ALL ■ starred questions only Flash review — 30 sec per question Close book and sleep early!	Rest = better recall

■ Pre-Interview Checklist

■ Understand Event Loop with mental model (not just definition)

■ Can explain Closure with your own analogy

■ Know difference between Promise.all / allSettled / race

■ Practiced fetch() + error handling from memory

■ Can explain 'this' in 4 different contexts

■ Know var vs let vs const (hoisting + scope)

■ Reviewed your own project — ready to explain challenges

■ Slept 7+ hours the night before ■

You've got this! ■

Confidence + Curiosity + Clear explanations = Dream Internship ■

Anand — you're ready. Go ace it!